

PlugSure CSMS v1.3.0 — Final Pre-Deployment Audit

Date: 29 September 2026

Package: `plugsure-v1.3.0.zip` (SHA-256 begins `393df3d5019ec558` ; this report and one test fix were added afterwards)

Audited build: production build (`npm run build` , processes run from `dist/`)

Environment: Node.js v22.22.2 · PostgreSQL 16 (both the production targets) · Ubuntu 24.04

Method: clean extraction of the package, `npm ci` from the lockfile, production build, fresh database migrated and seeded with the package's own scripts, the two-process stack (gateway + API) run from `dist/` as the restricted database role with row-level security in force, then every automated suite in the package.

Verdict

Ready for a supervised pilot. Public paid charging remains blocked on two provider accounts (a contracted QRIS acquirer and a WhatsApp/SMS provider for driver sign-in codes), as the Readiness Review stated. Nothing found in this audit changes that verdict; the findings of the security review are fixed and verified, and tenant isolation and roaming authentication have now been attacked directly.

Results

Check	Result
<code>npm ci</code> from the lockfile	clean
<code>npm run build</code> (production)	clean
Typecheck (both configurations)	clean
Migrations 001–042 on a fresh PostgreSQL 16	apply cleanly; re-run reports "up to date"
Stack from <code>dist/</code>	"API bridge engaged", "row-level security is in force" (both processes), "background workers started"
<code>npm test</code> (unit and database tests, against <code>plugsure_audit_fix</code>)	571/571 , 0 skipped

End-to-end suites, run back-to-back against the built stack:

Suite	Result	Note
<code>e2e:console</code>	96/96	
<code>e2e:isolation</code>	45/45	
<code>e2e:ocpi-auth</code>	9/9	first run failed on a suite bug (identity call), fixed in this package; rerun 9/9
<code>e2e:driver</code>	50/50	
<code>e2e:driver-plus</code>	46/46	
<code>e2e:queue</code>	19/19	
<code>e2e:reservation-fees</code>	12/12	
<code>e2e:api-sandbox</code>	38/38	in the batch 36/38 (revoked key answered 429: the per-IP invalid-key guard tripped by earlier suites); alone 38/38
<code>e2e:fleet-billing</code>	53/53	
<code>e2e:pricing</code>	24/24	
<code>e2e:pnc</code>	37/37	
<code>e2e:onboarding</code>	19/19	
<code>e2e:integrations</code>	22/22	
<code>e2e:payment-methods</code>	21/21	
<code>e2e:card-holds</code>	38/38	
<code>e2e:linked-wallets</code>	19/19	
<code>e2e:postpay</code>	40/40	
<code>e2e:field</code>	147/147	

Suite	Result	Note
e2e:ocpi	74/74	
e2e:ocpi-emsp	60/60	
e2e:ocpi-profiles	36/36	
e2e:sdk	23/23	needs <code>npm run sdk</code> first; then 23/23
e2e:v2x	21/21	
e2e:ocmf	20/20	
e2e:sandbox-2x	17/17	
e2e:brand	28/28	
e2e:apns	20/20	
e2e:live-activity	17/17	

Every suite passes. The two notes are properties of the test harness, not the product: run `npm run sdk` before `e2e:sdk`, and run `e2e:api-sandbox` alone or at least a minute after other suites (the invalid-key guard, `API_KEY_AUTH_FAILURES_PER_MIN`, is per IP and shared by every suite on the host).

What was verified during this engagement

Security properties probed on the running stack (details in `RELEASE-NOTES-v1.3.0.md`):

- **Tenant isolation:** row-level security on every business table, enforced for the restricted role; attacked directly by `e2e:isolation` (45/45: two operators, every resource type, reads, writes, ID swaps, audit log).
- **Authentication:** constant-time comparison of every secret (charger keys, bridge token, sessions, payment signatures, webhooks, WhatsApp verify token); sign-in lockout that never reveals whether an address has an account; roaming partner tokens (`e2e:ocpi-auth` 9/9).
- **Input handling:** no SQL injection path (all dynamic SQL whitelisted); path traversal blocked on every file route; list parameters validated (400, never 500); OCPP frames schema-validated, oversized frames cut off, hostile identities refused at the handshake.
- **Charger gateway:** per-charger message budget (20/s, burst 200; slowed, never dropped); field-conditions suite 147/147 including offline uploads, duplicate frames, clock skew, power loss.
- **Money:** payment notifications must state an amount (fail closed); underpayment refused; refunds and captures idempotent; the running cost during a charge equals the final bill to the rupiah (database-backed tests).
- **Deployment:** non-root container, database and internal ports bound to localhost, systemd sandboxing, no default secrets in Docker Compose, strict script policy on the console and printable pages.
- **Documentation:** Deployment Guide regenerated from corrected source; README status current; Readiness Review erratum recorded.

Defects found and fixed in this cycle

Severity	Defect	Fix
Medium	Docker Compose defaulted the PostgreSQL superuser password to <code>plugsure</code>	required, no default; test
Medium	A payment notification without an amount was booked as paid	fail closed, 422, critical alert; test
Medium	No per-charger OCPP message limit	token bucket, pacing not dropping; test
Medium	Inline script allowed on printable pages	strict <code>script-src 'self'</code> ; Print button reworked; test
Low	Negative <code>limit</code> gave a 500	shared parser, 400; test
Low	WhatsApp verify token compared with <code>===</code>	constant-time; test
Low	Firmware links never expired	valid while a campaign needs the image, +24 h; test
Low	Sign-in revealed whether an address had an account (message and timing)	one answer, equal work; test
—	Replaced APNs key falsely marked "refused" (found via a flaky test)	refusal only for the stored key, resend at once; test
—	Raised API key rate limit not applied at once	credited immediately; test
—	<code>npm test</code> silently skipped two test files	glob quoted
—	Running cost during a charge (gap)	built; one pricing path with the bill

Before go-live (in addition to the Deployment Guide checklist)

1. **Provider accounts (P0):** QRIS acquirer contract and credentials; WhatsApp/SMS provider for sign-in codes. Enter under Govern → Integrations; make one small real payment per enabled method.
2. `POSTGRES_PASSWORD` **is now required** in `.env` for Docker Compose. An existing install that relied on the default must set it to the password the database was created with, then change it.
3. **Rebuild the iOS app** from the new build kit for the lock-screen running cost. Older builds keep working without it.
4. **Brief support staff:** every failed sign-in now says "invalid email or password (after 5 failed attempts, sign-in pauses for 15 minutes)", including a paused account.
5. **Development seed account** is `ops@plugsure.com` ; production uses `create-admin` .
6. **Real charger hardware:** run `docs/ACCEPTANCE-v1.3.md` on at least one real unit; the simulator proves the protocol path, not the hardware.
7. **Edge and load, on staging:** Caddy live (TLS, office allow-list, on-demand certificates for white-label hosts, `X-Client-Cert-Fingerprint`); a load test towards the planned charger count with `GATEWAY_DIAG=verbose` , watching pool waits, event-loop delay and worker pass times.
8. **Backups:** nightly database and storage backups, one restore rehearsed.

Not covered

Load at scale, the Caddy edge and real hardware need real infrastructure (item 7 above). Plug & Charge was verified by code review and the sandbox suite (`e2e:pnc 37/37`), not attacked with a real PKI.

Sources: RELEASE-NOTES-v1.3.0.md (sections "Cost during the charge", "Security review fixes", "Isolation and roaming authentication"), docs/READINESS-REVIEW-v1.3.md, docs/DEPLOYMENT-GUIDE-v1.3.html. Test logs from the run of 29 September 2026.